

ejerciciop3.pdf



Anónimo



Sistemas Concurrentes y Distribuidos



2º Grado en Ingeniería Informática



**Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada**



MÁSTER EN

Inteligencia Artificial & Data Management

MADRID

Formamos
talento para un futuro
Sostenible

saber más



LLÉVATE 2 CLASES PRÁCTICAS GRATIS

Elige entre el permiso A1 o B

APROVECHA LA
SÚPER OFERTA



```
// -----  
//  
// Sistemas concurrentes y Distribuidos.  
// Práctica 3. Implementación de algoritmos distribuidos con MPI  
//  
// Archivo: prodcons2.cpp  
// Implementación del problema del productor-consumidor con  
// un proceso intermedio que gestiona un buffer finito y recibe peticiones  
// en orden arbitrario  
// (versión con un único productor y un único consumidor)  
//  
// Historial:  
// Actualizado a C++11 en Septiembre de 2017  
// -----  
  
#include <iostream>  
#include <thread> // this_thread::sleep_for  
#include <random> // dispositivos, generadores y distribuciones aleatorias  
#include <chrono> // duraciones (duration), unidades de tiempo  
#include <mpi.h>  
  
using namespace std;  
using namespace std::this_thread ;  
using namespace std::chrono ;  
  
const int  
    num_procesos_esperado = 9 ,  
    num_items              = 20,  
    tam_vector             = 1,  
    np                     = 1,  
    nc                     = 7,  
    id_productor           = 8 ,  
    id_buffer              = 7 ,  
    id_consumidor          = 5 ,  
    etiq_gordos            = 1,  
    etiq_delgados          = 2,  
    etiq_productor         = 3;  
  
//*****  
// plantilla de función para generar un entero aleatorio uniformemente  
// distribuido entre dos valores enteros, ambos incluidos  
// (ambos tienen que ser dos constantes, conocidas en tiempo de compilación)  
//-----  
  
template< int min, int max > int aleatorio()  
{  
    static default_random_engine generador( (random_device())() );  
    static uniform_int_distribution<int> distribucion_uniforme( min, max ) ;  
    return distribucion_uniforme( generador );  
}  
// -----  
// ptooducir produce los numeros en secuencia (1,2,3,...)  
// y lleva espera aleatorio
```

ESCANÉA EL QR
Llévate 2 clases prácticas
totalmente **Gratis**



```

void funcion_productor()
{
    int          peticion,
                valor_rec = 1 ;
    int valor_prod;
    MPI_Status estado ;

    while(true)
    {
        MPI_Recv ( &valor_rec, 1, MPI_INT, id_buffer, 0, MPI_COMM_WORLD,&estado
    );
        // producir valor
        cout << "                SE EMPIEZAN A PREPARAR 10 HAMBURGUESAS" << endl;
        sleep_for(seconds( aleatorio<5,10>())) );
        cout << "                SE HAN PREPARADO 10 HAMBURGUESAS" << endl;
        // enviar valor
        cout << "                REPONEDOR VA A ENVIAR 10 HAMBURGUESAS " << endl;
        MPI_Ssend( &valor_prod, 1, MPI_INT, id_buffer, etiq_productor,
MPI_COMM_WORLD );

    }
}
// -----

```

```

void funcion_consumidor(int id_propio)
{
    int          peticion,
                valor_rec = 1 ;
    MPI_Status estado ;

    while(true)
    {
        cout << "EL ALUMNO " << id_propio << " VA HACIA EL CAMION" << endl;
        sleep_for(seconds( aleatorio<5,10>())) );
        cout << "EL ALUMNO " << id_propio << " HA LLEGADO AL CAMION" << endl;
        cout << "EL ALUMNO " << id_propio << " VA A PEDIR UNA HAMBURGUESA" <<
endl;

        if(id_propio % 2 == 0){

            MPI_Ssend( &peticion, 1, MPI_INT, id_buffer, etiq_gordos,
MPI_COMM_WORLD);

        }else{

            MPI_Ssend( &peticion, 1, MPI_INT, id_buffer, etiq_delgados,
MPI_COMM_WORLD);

        }

        MPI_Recv ( &valor_rec, 1, MPI_INT, id_buffer, 0, MPI_COMM_WORLD,&estado );
        if(id_propio % 2 == 0){

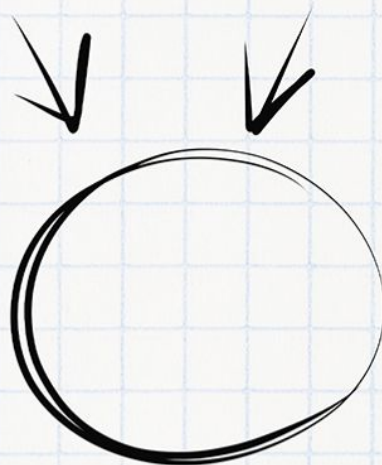
```

Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?

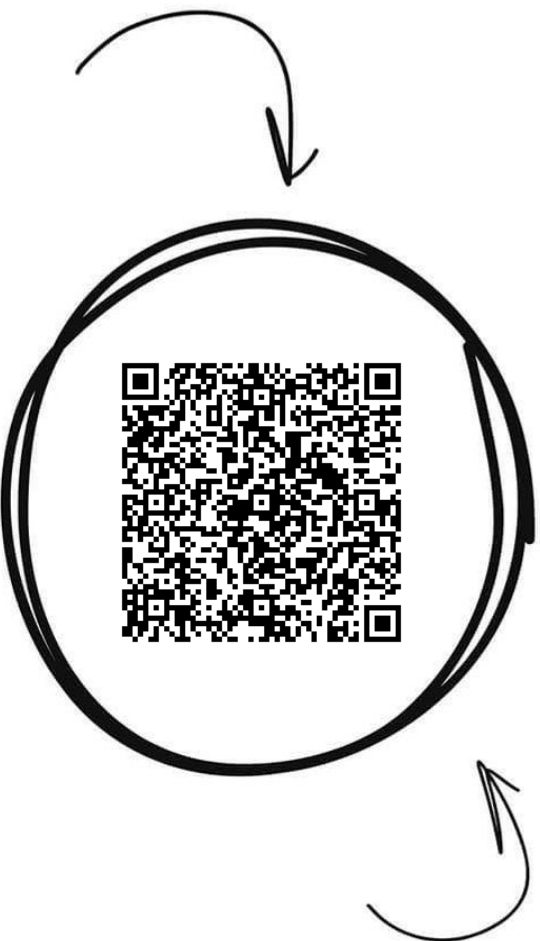


WUOLAH

Sistemas Concurrentes y Dist...



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

WUOLAH

- 1** Imprime esta hoja
- 2** Recorta por la mitad
- 3** Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

- 4** Llévate dinero por cada descarga de los documentos descargados a través de tu QR



```

        cout << "Alumno " << id_propio << " ha recibido 2 hamburguesas " << endl
<< flush ;
    }else{

        cout << "Alumno " << id_propio << " ha recibido 1 hamburguesa " << endl
<< flush ;

    }

    cout << "EL ALUMNO " << id_propio << " EMPIEZA A COMER Y SE ALEJA DEL
CAMION" << endl;
    sleep_for(seconds( aleatorio<5,10>()) );

    }
}
// -----

void funcion_buffer()
{

    MPI_Status estado ;                // metadatos del mensaje recibido
    int disponibles = 10;
    int valor;
    int peticion;

    while(true)
    {
        // 1. determinar si puede enviar solo prod., solo cons, o todos

        cout << "          " << disponibles << " ES EL NUMERO DE HAMBURGUESAS
DISPONIBLES" << endl;
        if ( disponibles >= 5 ){

            MPI_Recv( &valor, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG,
MPI_COMM_WORLD, &estado );

            }else if(disponibles > 1){

            MPI_Recv( &valor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_delgados,
MPI_COMM_WORLD, &estado );

            }else{

            MPI_Ssend( &peticion, 1, MPI_INT, id_productor, 0, MPI_COMM_WORLD);
            MPI_Recv( &valor, 1, MPI_INT, id_productor, etiq_productor,
MPI_COMM_WORLD, &estado );

            }

        switch( estado.MPI_TAG) // leer emisor del mensaje en metadatos
        {
            case 3: // si ha sido el productor: insertar en buffer

```

LLÉVATE 2 CLASES PRÁCTICAS GRATIS

Elige entre el permiso A1 o B

APROVECHA LA
SÚPER OFERTA



```
        disponibles = 10;

        break;

    case 2: // si ha sido el consumidor: extraer y enviarle

        disponibles --;

        cout << "Dependiente va a enviar una hamburguesa al alumno " <<
estado.MPI_SOURCE << endl ;
        MPI_Ssend( &valor, 1, MPI_INT, estado.MPI_SOURCE, 0,
MPI_COMM_WORLD);
        break;

    case 1:

        disponibles -= 2;

        cout << "Dependiente va a enviar dos hamburguesas al alumno " <<
estado.MPI_SOURCE << endl ;
        MPI_Ssend( &valor, 1, MPI_INT, estado.MPI_SOURCE, 0, MPI_COMM_WORLD);
        break;

    }
}

// -----

int main( int argc, char *argv[] )
{
    int id_propio, num_procesos_actual;

    // inicializar MPI, leer identif. de proceso y número de procesos
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &id_propio );
    MPI_Comm_size( MPI_COMM_WORLD, &num_procesos_actual );

    if ( num_procesos_esperado == num_procesos_actual )
    {
        // ejecutar la operación apropiada a 'id_propio'
        if ( id_propio == id_productor )
            funcion_productor();
        else if ( id_propio == id_buffer )
            funcion_buffer();
        else

            funcion_consumidor(id_propio);
    }
    else
    {
        if ( id_propio == 0 ) // solo el primero escribe error, indep. del rol
        { cout << "el número de procesos esperados es: " <<
num_procesos_esperado << endl
```

ESCANÉA EL QR

Llévate 2 clases prácticas
totalmente **Gratis**



```
        << "el número de procesos en ejecución es: " << num_procesos_actual
<< endl
        << "(programa abortado)" << endl ;
    }
}

// al terminar el proceso, finalizar MPI
MPI_Finalize( );
return 0;
}
```